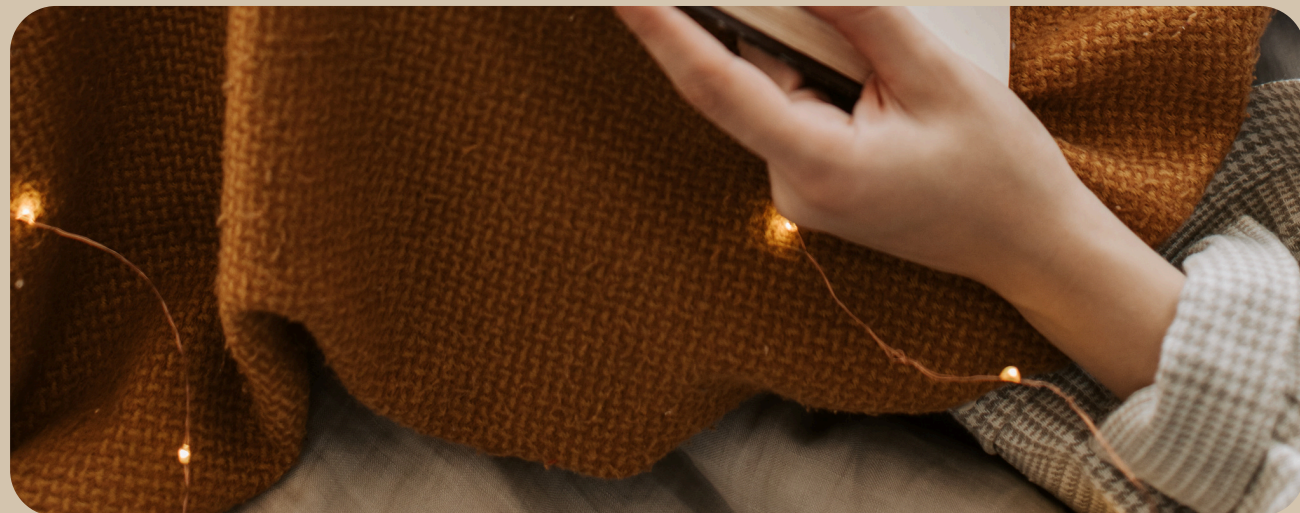


DIGITAL LIBRARY MANAGEMENT USING BST



LANGUAGE: DATASTRUCTURES
TOPIC: BINARY SEARCH TREE OPERATIONS
PRESENTED BY: N. VARSHINI
P.MANOJNA



- Introduction
- A Digital Library stores book genres
- Uses Binary Search Tree (BST)
- Efficient searching, insertion, deletion
- Data is stored in sorted order
- What is BST?
- BST = Binary Search Tree
- Each node has:
- Left child (smaller values)
- Right child (greater values)
- Helps in fast operations ($O(\log n)$)

Structure of Node

```
typedef struct Node {  
    char genre[50];  
    struct Node *left;  
    struct Node *right;  
} Node;
```

- Stores genre name
- Left and right pointers

* Create Node

```
Node* createNode(char *genre) {  
    Node* newNode = (Node*)malloc(sizeof(Node));  
    strcpy(newNode->genre, genre);  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

- Allocates memory
- Initializes node





*Insert Operation

- Adds a new genre
- Follows BST rules:
 - Smaller → left
 - Greater → right

```
root = insert(root, genre);
```



*Search Operation

- Finds a genre
- Compares using strcmp()

```
if (search(root, genre))  
    printf("Found");
```

- Fast searching using BST



* Delete Operation

- Removes a node
- 3 cases:
 - No child
 - One child
 - Two children
- Uses findMin() for replacement

* Update & Display

Update:

- Search old genre
- Replace with new genre

Display:

- Inorder traversal
- Prints sorted genres

```
display(root);
```





Conclusion

- BST improves efficiency
- Supports all operations:
 - Insert
 - Delete
 - Search
 - Update
- Useful in real-world applications like libraries



THANK YOU